

LAPORAN PRATIUM STRUKTUR DATA

IMPLEMENTASI QUEUE DAN PRINSIP FIFO DALAM JAVA

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum "Implementasi Queue dan Prinsip FIFO dalam Java" ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Queue yang bekerja dengan prinsip FIFO (First In, First Out), serta penerapannya melalui implementasi menggunakan library bawaan Java (LinkedList), array statis sirkular (circular array), dan studi kasus nyata seperti simulasi antrian loket dan pembalikan urutan data menggunakan kombinasi Stack-Queue. Saya menyadari bahwa pemahaman tentang struktur data antrian merupakan fondasi penting dalam penjadwalan proses, algoritma pencarian lebar (BFS), manajemen buffer, dan pengembangan sistem yang memprioritaskan urutan kedatangan

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya supaya laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Queue dalam Java	5
2.2 Langkah pengerjaan	6
2.3 Latihan	11
BAB III KESIMPULAN.....	13
3.1 Kesimpulan	13
3.2 Saran	13
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, tidak semua masalah dapat diselesaikan dengan struktur data yang memproses elemen secara acak atau berdasarkan posisi terakhir masuk. Banyak sistem nyata justru mengharuskan data diproses sesuai urutan kedatangan, di mana elemen yang pertama masuk harus diproses lebih dahulu. Prinsip ini dikenal sebagai FIFO (First In, First Out) dan diimplementasikan melalui struktur data Queue (Antrian).

Berbeda dengan Stack pada praktikum sebelumnya yang membatasi akses hanya pada ujung atas (top), Queue membatasi operasi pada dua ujung: rear (belakang) untuk penyisipan dan front (depan) untuk penghapusan. Pembatasan ini sangat berguna dalam skenario yang memerlukan keadilan urutan, seperti penjadwalan CPU, antrian cetak (printer spooling), buffer streaming, dan algoritma traversing graf (BFS). Pada praktikum ini, mahasiswa diminta mengimplementasikan Queue melalui berbagai pendekatan (`java.util.Queue`, array sirkular statis, dan `LinkedList`), serta menerapkannya pada studi kasus seperti pembalikan data dan simulasi antrian layanan.

1.2 Tujuan

1. Memahami prinsip kerja struktur data Queue dan konsep FIFO.
2. Mengimplementasikan Queue menggunakan interface `java.util.Queue` dengan backing class `LinkedList`.
3. Membangun ADT Queue secara manual menggunakan Array Statis Sirkular (Circular Array).
4. Menerapkan Queue untuk menyelesaikan masalah algoritma seperti pembalikan urutan data dan simulasi antrian layanan.
5. Menganalisis kompleksitas waktu dan ruang dari setiap operasi dasar Queue (`enqueue`, `dequeue`, `peek`, `isEmpty`).

1.3 Manfaat

1. Mahasiswa mampu membedakan karakteristik Queue dengan struktur data linier lainnya seperti Stack dan Array.
2. Mahasiswa memahami pentingnya penggunaan array sirkular untuk menghindari pergeseran data (shifting) yang tidak efisien pada antrian statis.
3. Mahasiswa memperoleh pengalaman dalam menerjemahkan logika antrian dunia nyata (loket, printer, buffer) ke dalam implementasi kode yang robust.
4. Melatih kemampuan debugging dan penanganan kondisi batas seperti Queue Overflow dan Queue Underflow.

BAB II

PEMBAHASAN

2.1 Konsep Stack dalam Java

Queue adalah koleksi objek yang terurut secara linier dengan aturan penyisipan (enqueue) pada ujung belakang (rear) dan penghapusan (dequeue) pada ujung depan (front). Operasi fundamental pada Queue meliputi:

- enqueue(e) / add(e) / offer(e): Menambahkan elemen e ke bagian belakang queue.
- dequeue() / remove() / poll(): Mengambil dan menghapus elemen terdepan.
- peek() / element(): Melihat elemen teratas tanpa menghapusnya.
- isEmpty(): Memeriksa apakah queue kosong.
- size(): Mengembalikan jumlah elemen.

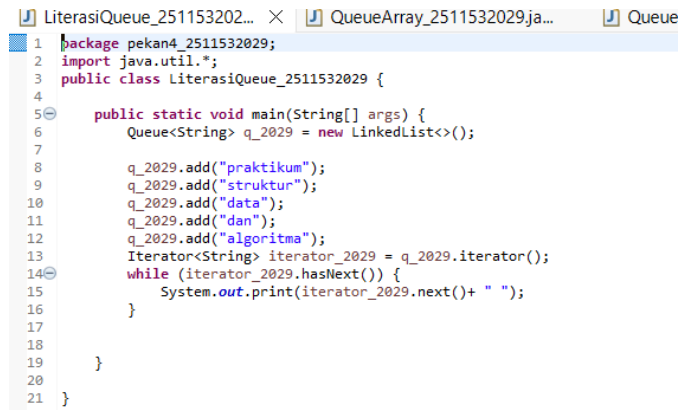
Dalam Java, Queue dapat diimplementasikan melalui:

1. java.util.Queue interface dengan backing class LinkedList atau ArrayDeque (modern, non-thread-safe, namun lebih cepat dari legacy Vector).
2. Implementasi manual menggunakan Array Statis Sirkular (Circular Array) untuk memahami mekanisme penunjuk front, rear, dan operator modulo %.

2.2 Langkah pengerjaan

1. LiterasiQueue_2511532029

Program:



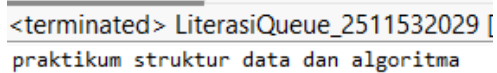
```
1 package pekan4_2511532029;
2 import java.util.*;
3 public class LiterasiQueue_2511532029 {
4
5     public static void main(String[] args) {
6         Queue<String> q_2029 = new LinkedList<>();
7
8         q_2029.add("praktikum");
9         q_2029.add("struktur");
10        q_2029.add("data");
11        q_2029.add("dan");
12        q_2029.add("algoritma");
13        Iterator<String> iterator_2029 = q_2029.iterator();
14        while (iterator_2029.hasNext()) {
15            System.out.print(iterator_2029.next()+ " ");
16        }
17
18    }
19
20 }
21 }
```

Gambar 2.1

Penjelasan program:

- 1) `Queue<String> q_2029 = new LinkedList<>();` → Mendeklarasikan queue dengan implementasi `LinkedList` yang secara otomatis mendukung antarmuka `FIFO`.
- 2) `q_2029.add(...)` → Menambahkan 5 string secara berurutan. Sesuai prinsip `FIFO`, "praktikum" menjadi elemen terdepan dan "algoritma" menjadi elemen terbelakang.
- 3) `Iterator<String> iterator_2029 = q_2029.iterator();` → Melakukan iterasi menggunakan iterator bawaan Java untuk mencetak elemen tanpa menghapusnya.
- 4) **Analisis Kompleksitas:** Waktu $O(n)$ untuk iterasi dan pencetakan. Ruang $O(n)$ untuk penyimpanan elemen. Operasi `add()` dan `peek()` pada `LinkedList` memiliki kompleksitas $O(1)$.

Output :



```
<terminated> LiterasiQueue_2511532029 [
praktikum struktur data dan algoritma
```

Gambar 2.2

2. Program QueueArray_2511532029 & QueueArrayDriver_2511532029 Program:

```

1 package pekand_2511532029;
2
3 public class QueueArray_2511532029 {
4     int front_2029, rear_2029, size_2029;
5     int capacity_2029;
6     int array_2029[];
7
8     public QueueArray_2511532029(int capacity_2029) {
9         this.capacity_2029 = capacity_2029;
10        front_2029 = this.size_2029 = 0;
11        rear_2029 = capacity_2029 - 1;
12        array_2029 = new int [this.capacity_2029];
13    }
14
15    boolean isFull_2029 (QueueArray_2511532029 queue) {
16        return (queue.size_2029 == queue.capacity_2029);
17    }
18
19    boolean isEmpty_2029 (QueueArray_2511532029 queue) {
20        return (queue.size_2029 == 0);
21    }
22
23    void enqueue_2029 (int item_2029) {
24        if (isFull_2029(this)) return;
25
26        this.rear_2029 = (this.rear_2029 + 1) % this.capacity_2029;
27        this.array_2029 [this.rear_2029] = item_2029;
28        this.size_2029 = this.size_2029 + 1;
29        System.out.println(item_2029 + "enqueue to queue");
30    }
31
32    int dequeue_2029() {
33        if (isEmpty_2029(this))
34            return Integer.MIN_VALUE;
35        int item_2029 = this.array_2029[this.front_2029];
36        this.front_2029 = (this.front_2029 + 1) % this.capacity_2029;
37        this.size_2029 = size_2029 - 1;
38        return item_2029;
39    }
40
41    int front_2029 () {
42        if (isEmpty_2029(this))
43            return Integer.MIN_VALUE;
44        return this.array_2029[this.front_2029];
45    }
46
47    int rear_2029() {
48        if (isEmpty_2029(this))
49            return Integer.MIN_VALUE;
50        return this.array_2029[this.rear_2029];
51    }
52
53    void display_2029() {
54        int i_2029;
55        if (front_2029 == rear_2029) {
56            System.out.println("\n Antrian Kosong\n");
57            return;
58        }
59        for (i_2029 = front_2029; i_2029 < rear_2029; i_2029++) {
60            System.out.printf("%d <- ", array_2029[i_2029]);
61        }
62        return;
63    }
64
65 }
66

```

Gambar 2.3

```

1 package pekand_2511532029;
2
3 public class QueueArrayDriver_2511532029 {
4
5     public static void main(String[] args) {
6         QueueArray_2511532029 queue_2029 = new QueueArray_2511532029(1000);
7         queue_2029.enqueue_2029(10);
8         queue_2029.enqueue_2029(20);
9         queue_2029.enqueue_2029(30);
10        queue_2029.enqueue_2029(40);
11        System.out.println("item didepan " + queue_2029.front_2029());
12        System.out.println("item paling belakang " + queue_2029.rear_2029());
13        System.out.println("Tampilkan queue");
14        queue_2029.display_2029();
15        System.out.println();
16        System.out.println(queue_2029.dequeue_2029() + "Dihapus dari queue");
17        System.out.println("item didepan " + queue_2029.front_2029());
18        System.out.println("item dibelakang " + queue_2029.rear_2029());
19        System.out.println("Tampilkan queue setelah satu data dihapus");
20        queue_2029.display_2029();
21    }
22
23 }

```

Gambar 2.4

Penjelasan program:

1. `int front_2029, rear_2029, size_2029; int array_2029[];` → Implementasi berbasis array statis sirkular. `front` menunjuk ke elemen terdepan, `rear` ke posisi penambahan terakhir.
2. `this.rear_2029 = (this.rear_2029 + 1) % this.capacity_2029;` → Penggunaan operator modulo memastikan indeks `rear` berputar kembali ke 0 saat mencapai batas array, mencegah waste memory.
3. `enqueue_2029() & dequeue_2029()` → Mengecek `isFull/isEmpty` untuk mencegah Overflow/Underflow. `front` juga digeser dengan modulo saat `dequeue`.
4. `QueueArrayDriver_2029` → Menguji urutan FIFO: enqueue 10, 20, 30, 40 → `front` menunjukkan 10, `rear` menunjukkan 40 → `dequeue` mengembalikan 10 → `front` bergeser ke 20.
5. Analisis Kompleksitas: Seluruh operasi dasar (`enqueue`, `dequeue`, `front`, `rear`) memiliki kompleksitas waktu $O(1)$ dan ruang $O(1)$ tambahan. Array statis memberikan overhead memori yang minimal.

Output:

```
<terminated> QueueArrayDriver_2511532029 (Java Applica
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
item di depan 10
item paling belakang 40
Tampilan queue
10 <--20 <--30 <--
100ihapus dari queue
item di depan1
item di belakang3
Tampilan queue setelah satu data dihapus
20 <--30 <--
```

3. Program QueueLinkedList_2029

Program:

```
1 package pekan4_2511532029;
2
3 import java.util.Queue;
4
5 public class QueueLinkedList_2511532029 {
6
7     public static void main(String[] args) {
8         Queue<Integer> q_2029 = new LinkedList<>();
9
10         for (int i_2029 = 0; i_2029 < 6; i_2029++)
11             q_2029.add(i_2029);
12         System.out.println("Elemen Antrean" + q_2029);
13
14         int hapus_2029 = q_2029.remove();
15         System.out.println("Hapus elemen = " + hapus_2029);
16         System.out.println(q_2029);
17
18         int depan_2029 = q_2029.peek();
19         System.out.println("Kepala antrean = " + depan_2029);
20
21         int banyak_2029 = q_2029.size();
22         System.out.println("Size antran = " + banyak_2029);
23
24     }
25 }
26
27 }
```

Gambar 2.4

Penjelasan program:

- 1) `Queue<Integer> q_2029 = new LinkedList<>();` → Menggunakan library bawaan Java untuk queue dinamis.
- 2) `for (int i_2029 = 0; i_2029 < 6; i_2029++) q_2029.add(i_2029);` → Menambahkan angka 0-5 secara berurutan.
- 3) `q_2029.remove()` → Menghapus dan mengembalikan elemen terdepan (0).
- 4) `q_2029.peek() & q_2029.size()` → Melihat kepala antrian baru (1) dan jumlah sisa elemen (5).
- 5) Program ini mendemonstrasikan kemudahan penggunaan ADT bawaan Java yang menangani alokasi memori dinamis secara otomatis.

Output:

```
<terminated> QueueLinkedList_2511532029!  
Elemen Antrean[0, 1, 2, 3, 4, 5]  
Hapus elemen = 0  
[1, 2, 3, 4, 5]  
Kepala antrean = 1  
Size antran =5
```

Gambar 2.6

4. Program ReverseData_2511532029

Program:

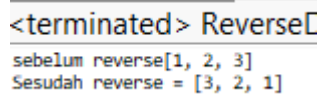
```
1 package pekan3_25115302029;  
2 import java.util.Scanner;  
3  
4 public class stackPostfix_2511532029 {  
5     public static int postfixEvaluate_2029(String expression)  
6     {  
7         Stack<Integer> s_2029 = new Stack<Integer>();  
8         Scanner input_2029 = new Scanner (expression);  
9         while (input_2029.hasNext()) {  
10             if (input_2029.hasNextInt()){  
11                 s_2029.push(input_2029.nextInt());  
12             }else {  
13                 String operator_2029 = input_2029.next();  
14                 int operand2_2029 = s_2029.pop();  
15                 int operand1_2029 = s_2029.pop();  
16                 if (operator_2029.equals("+")) {  
17                     s_2029.push(operand1_2029 + operand2_2029);  
18                 } else if (operator_2029.equals("-")) {  
19                     s_2029.push(operand1_2029 - operand2_2029);  
20                 } else if (operator_2029.equals("*")) {  
21                     s_2029.push(operand1_2029 * operand2_2029);  
22                 } else {  
23                     s_2029.push(operand1_2029/operand2_2029);  
24                 }  
25             }  
26         }  
27         input_2029.close();  
28         return s_2029.pop();  
29     }  
30  
31  
32     public static void main(String[] args) {  
33         System.out.println("HASIL postfix ="+ postfixEvaluate_2029("5 2 4 * + 7 -"));  
34     }  
35 }  
36 }
```

Gambar 2.7

Penjelasan program:

1. `Queue<Integer> q_2029 = new LinkedList<>();` → Inisialisasi queue dengan data 1, 2, 3.
2. `while(!q_2029.isEmpty()) { s_2029.push(q_2029.remove()); }` → Memindahkan seluruh elemen dari queue ke stack. Karena stack bersifat LIFO, urutan terbalik.
3. `while(!s_2029.isEmpty()) { q_2029.add(s_2029.pop()); }` → Memindahkan kembali dari stack ke queue. Hasilnya adalah queue dengan urutan terbalik (3, 2, 1).
4. Studi kasus ini membuktikan bagaimana kombinasi ADT (Queue + Stack) dapat menyelesaikan masalah manipulasi urutan data secara elegan.
5. **Analisis Kompleksitas:** Waktu $O(n)$, Ruang $O(n)$ untuk stack bantuan.
Output: sebelum reverse [1, 2, 3] → Sesudah reverse = [3, 2, 1]

Output:



```
<terminated> ReverseC
sebelum reverse[1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

Gambar 2.8

2.3 Latihan

Deskripsi Program:

Program ini mensimulasikan sistem antrian loket pelayanan menggunakan prinsip FIFO. Data pelanggan disimpan dalam array sirkular bertipe String.

1.AntrianLoket 2511532029

```
1 package pekani_2511532029;
2
3 import java.util.Scanner;
4
5 public class AntrianLoket_2511532029 {
6     private String[] queue_2029;
7     private int front_2029;
8     private int rear_2029;
9     private int max_2029;
10    private int size_2029;
11
12    public AntrianLoket_2511532029(int kapasitas) {
13        max_2029 = kapasitas;
14        queue_2029 = new String[max_2029];
15        front_2029 = 0;
16        rear_2029 = -1;
17        size_2029 = 0;
18    }
19
20    public boolean isEmpty() {
21        return size_2029 == 0;
22    }
23
24    public boolean isFull() {
25        return size_2029 == max_2029;
26    }
27
28    public void enqueue(String data) {
29        if (isFull()) {
30            System.out.println("Antrian penuh! Tidak dapat menambahkan data.");
31            return;
32        }
33        rear_2029 = (rear_2029 + 1) % max_2029;
34        queue_2029[rear_2029] = data;
35        size_2029++;
36        System.out.println("Data berhasil ditambahkan ke antrian");
37    }
38
39    public String dequeue() {
40        if (isEmpty()) {
41            System.out.println("Antrian kosong! Tidak ada data yang dapat dihapus.");
42            return null;
43        }
44        String dataDihapus = queue_2029[front_2029];
45        front_2029 = (front_2029 + 1) % max_2029;
46        size_2029--;
47        System.out.println(dataDihapus + " telah dilayani");
48        return dataDihapus;
49    }
50
51    public void display() {
52        if (isEmpty()) {
53            System.out.println("Antrian kosong.");
54            return;
55        }
56        System.out.println("Isi antrian:");
57        for (int i = 0; i < size_2029; i++) {
58            int idx = (front_2029 + i) % max_2029;
59            System.out.println((i + 1) + ". " + queue_2029[idx]);
60        }
61    }
62
63    public void reverse() {
64        if (isEmpty()) {
65            System.out.println("Antrian kosong, tidak ada yang dibalik.");
66            return;
67        }
68        String[] temp = new String[size_2029];
69        for (int i = 0; i < size_2029; i++) {
70            temp[i] = queue_2029[(front_2029 + i) % max_2029];
71        }
72        for (int i = 0; i < size_2029 / 2; i++) {
73            String t = temp[i];
74            temp[i] = temp[size_2029 - 1 - i];
75            temp[size_2029 - 1 - i] = t;
76        }
77        front_2029 = 0;
78        rear_2029 = size_2029 - 1;
79        for (int i = 0; i < size_2029; i++) {
80            queue_2029[i] = temp[i];
81        }
82        System.out.println("Antrian berhasil dibalik.");
83    }
84
85    public static void main(String[] args) {
86        Scanner scanner = new Scanner(System.in);
87        AntrianLoket_2511532029 antrian = new AntrianLoket_2511532029(5);
88        int pilihan;
89
90        System.out.println("==== PROGRAM ANTRIAN LOKET ===");
91        do {
92            System.out.println("1. Tambah Antrian");
93            System.out.println("2. Hapus Antrian");
94            System.out.println("3. Tampilkan Antrian");
95            System.out.println("4. Reverse");
96            System.out.println("5. Keluar");
97            System.out.print("Pilih menu: ");
98            pilihan = scanner.nextInt();
99            scanner.nextLine();
100
101            switch (pilihan) {
102                case 1:
103                    System.out.print("Masukkan nama pelanggan: ");
104                    String nama = scanner.nextLine();
105                    antrian.enqueue(nama);
106                    break;
107                case 2:
108                    antrian.dequeue();
109                    break;
110                case 3:
111                    antrian.display();
112                    break;
113                case 4:
114                    antrian.reverse();
115                    antrian.display();
116                    break;
117                case 5:
118                    System.out.println("Program selesai");
119                    break;
120                default:
121                    System.out.println("Pilihan tidak valid.");
122            }
123        } while (pilihan != 5);
124
125        scanner.close();
126    }
127 }
```

Gambar 2.10

a) Deskripsi Logika & Fitur:

- Struktur Data: Array statis `queue_2029[]` dengan penunjuk `front_2029`, `rear_2029`, dan counter `size_2029`.
- `enqueue(String data)`: Menambahkan nama pelanggan ke posisi `rear`. Menggunakan modulo $(rear + 1) \% max$ untuk sirkularitas. Mengecek `isFull()` untuk keamanan.
- `dequeue()`: Menghapus dan mengembalikan pelanggan di posisi `front`. Menggeser `front` dengan modulo. Mengecek `isEmpty()`.
- `display()`: Mencetak isi antrian dari `front` hingga `rear` menggunakan loop $(front + i) \% max$.
- `reverse()`: Membalik urutan antrian menggunakan array sementara `temp[]`. Elemen disalin, dibalik dengan teknik dua pointer, lalu dikembalikan ke array utama. Pointer `front` dan `rear` direset ke posisi linear.
- Menu Interaktif: Menggunakan `do-while` dan `switch-case` agar program terus berjalan hingga user memilih keluar.

b) Analisis Kompleksitas:

- `enqueue`, `dequeue`, `front`, `rear`: $O(1)$
- `display`: $O(n)$
- `reverse`: $O(n)$ waktu dan ruang (karena alokasi array sementara)
Program berhasil menangani kondisi batas (antrian penuh/kosong) dan menunjukkan fleksibilitas array sirkular dalam simulasi layanan nyata.

Output:

```
<terminated> AntrianLoket_2511532029 [Java A
=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: iqbal abin
Data berhasil ditambahkan ke antrian

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: pasya abin
Data berhasil ditambahkan ke antrian

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. iqbal abin
2. pasya abin

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik.
Isi antrian:
1. pasya abin
2. iqbal abin

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
pasya abin telah dilayani

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
iqbal abin telah dilayani

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

Gambar 2.11

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum implementasi Queue dalam Java, dapat disimpulkan bahwa Queue merupakan struktur data linier yang bekerja dengan prinsip FIFO (First In, First Out), di mana penyisipan dilakukan di ujung belakang (rear) dan penghapusan dilakukan di ujung depan (front). Struktur ini dapat diimplementasikan melalui berbagai pendekatan, mulai dari interface `java.util.Queue` dengan `LinkedList` untuk kapasitas dinamis, hingga array statis sirkular yang menawarkan efisiensi memori tinggi dan menghindari pergeseran data. Seluruh operasi dasar Queue seperti enqueue, dequeue, dan peek memiliki kompleksitas waktu $O(1)$, menjadikannya sangat optimal untuk skenario penjadwalan dan pemrosesan berurutan.

Penerapan konsep Queue telah berhasil divalidasi melalui studi kasus pembalikan data menggunakan bantuan Stack dan simulasi antrian loket pelayanan, yang membuktikan relevansinya dalam sistem layanan, buffer I/O, dan algoritma graf. Selain itu, praktikum ini menekankan pentingnya penggunaan operator modulo pada array sirkular serta penanganan kondisi overflow/underflow guna mencegah `NullPointerException` dan menjamin stabilitas program.

3.2 Saran

1. Disarankan untuk menambahkan materi mengenai `PriorityQueue` dan `Deque` pada pertemuan selanjutnya, agar mahasiswa memahami variasi queue yang mendukung prioritas dan operasi dua ujung.
2. Sebaiknya praktikum dilengkapi dengan studi kasus visual atau integrasi GUI sederhana, agar mahasiswa dapat melihat secara langsung bagaimana queue berputar dan berubah state secara real-time.
3. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang sebelum menulis kode di laboratorium.

DAFTAR PUSTAKA

[1] Oracle. "Queue (Java Platform SE 17)". The Java™ Tutorials. 2024. Available:
Available:
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html>